This tool is an advanced AI-powered debate suite designed to elevate the way debaters practice, prepare, and improve. It integrates three core functionalities that together form a comprehensive ecosystem for competitive debate preparation and skill refinement.

Seamless User Experience & Centralized Dashboard

From the moment a user logs in through the authentication page—either by creating a new account or accessing an existing one—they are immersed in a unified dashboard. This centralized hub allows debaters to manage their workflow: tracking tasks related to upcoming tournaments, engaging with a conversational AI chatbot for quick questions or sparring, and storing judge feedback from past competitions. By compiling all debate-related activities and materials in one place, users can stay organized, focused, and efficient.

Realistic AI-Driven Mock Debates

On the main page, users can engage in AI-assisted debate simulations, practicing in 2v1 formats or with an AI partner for 2v2. This flexibility allows students to simulate real-world tournament conditions—even without a team present. These mock debates are not just casual sparring; they are structured and generate downloadable content including transcripts, speeches, and flowcharts. This enables students to reflect deeply on their arguments, delivery, and structure outside of the live practice environment.

Rich Feedback & Analytical Tools

The analysis page transforms raw practice data into actionable insight. Users can upload recordings or files from both AI debates and in-person rounds, receiving tailored feedback that highlights strengths, weaknesses, logical inconsistencies, and stylistic opportunities for improvement. This level of diagnostic support empowers debaters to take ownership of their growth by understanding not just what went wrong—but why.

Strategic Case Building with AI Assistance

Recognizing that feedback without implementation is wasted potential, the platform includes powerful tools to help debaters construct and refine their affirmative (Aff) and negative (Neg) cases. Leveraging AI-generated suggestions, users can adapt their argumentation style, deepen evidence bases, and strategically tweak frameworks. This turns passive feedback into active development and gives users the confidence that their cases are battle-tested and intelligently built.

The Impact: Transforming Debate Preparation at Scale

The implications of this tool extend far beyond convenience. Here's how it stands to reshape the debate landscape:

- **Accessibility & Equity:** Students from under-resourced schools or without access to experienced coaches can now receive world-class practice and feedback anytime, anywhere.

- **Continuous Growth:** With instant analysis and AI partners always available, debaters can practice more frequently, iterating on their arguments faster than ever before.

- **Coach Empowerment:** Coaches can focus on higher-level instruction, knowing that their students have tools to independently review feedback, test cases, and simulate rounds.

- **Tournament Readiness:** By mimicking tournament structures and storing real feedback, students approach real rounds with more preparation, confidence, and strategic clarity.

- **Community Development:** As more debaters use the platform, it can foster a growing, data-driven community where strategies evolve, ideas circulate, and competition becomes more about skill than resources.

In essence, this is not just a debate tool—it's a digital debate partner, mentor, and strategist rolled into one. By integrating AI into every stage of the preparation process, it has the potential to revolutionize how debate is taught, practiced, and perfected.

To run this project locally, copy the entire codebase into a code editor of your choice (such as WebStorm, Visual Studio Code, or any other IDE). After setting up your environment:

1. **Install dependencies**: Use the *requirements.txt* file to install all the necessary Python libraries by running *pip install -r requirements.txt.*

2. **Run the application**: Launch the app by executing *python app.py*. This should open a local development server and launch the web app in your default browser.

Important Note on API Usage:
The application uses OpenRouter's AI API to provide intelligent analysis and feedback. If your current API key is disabled due to a lack of credits, you can create a new account on OpenRouter, generate a new key, and update the `OPENROUTER_API_KEY` variable inside `app.py`.

The frameworks utilized were frontend technologies (HTML, CSS, and JS) with jinja templating, tailwindCSS, and Quill.js for styling. The backend was created using python + flask + SQLite alongside frameworks such as werkzeug for user authentication, requests for OpenRouter AI APIs, and python-docx for seamless file reading.

Now, the primary skills that I have been attempting to develop have been my front-end skills. Throughout this project, I have practiced creating interactive UI elements such as dropdowns, modals, and animations that elevate the UI. I have also expanded upon my styling skills through spending extra time to make sure everything in the UI looks great. Moreover, I have also learned many more different flask methods, improving my skills there. I have also expanded upon my werkzeug.security skills. Lastly, it was my first time utilizing python.docx, a skill that unlocks new horizons in my ability to enable user file storage and reading.

Tech Stack

The project integrates both frontend and backend technologies to deliver a seamless, full-stack application experience.

On the **backend**, I used **Python with Flask** as the core framework for routing, user authentication, and handling database interactions. The application uses **SQLite** as a lightweight yet effective solution for storing user data, uploaded documents, and analysis results. For secure user management, I integrated **Werkzeug**, which handles password hashing and session authentication. The **Requests** library was essential for communicating with the OpenRouter AI API, allowing the application to send user data and receive intelligent, AI-generated feedback. Lastly, I employed **python-docx**, which allowed the application to read and parse `.docx` files uploaded by users, enabling dynamic content extraction and analysis.

On the **frontend**, I worked with **HTML, CSS, and JavaScript** to build the core structure and interactivity of the web pages. I used **Jinja2**, Flask's built-in templating engine, to dynamically inject content into HTML files based on user sessions and uploaded content. **Tailwind CSS** played a crucial role in styling the application with its utility-first classes, helping me achieve a modern, responsive design without writing extensive custom CSS. I also incorporated **Quill.js**, a powerful rich-text editor, to allow users to format their writing and interact with editable content in a polished, intuitive interface.

Key Skills Developed

Throughout this project, I made a deliberate effort to strengthen my **frontend engineering** abilities while also expanding my backend development expertise. I practiced integrating interactive user interface elements, learned new backend frameworks and methods, and gained experience working with file parsing and API integration. These new skills will be invaluable in future hackathons and in scaling my existing projects.

✅ 1. Frontend Design & Interaction (Tailwind CSS + JS + Quill.js)

One of the key areas I focused on was frontend interaction. I learned how to create interactive components such as dropdowns, modals, and animations that not only enhanced usability but also gave the application a polished look. I deepened my understanding of **Tailwind CSS**, learning how to apply responsive design principles through utility classes, enabling mobile-friendly and accessible designs. Additionally, I integrated and configured **Quill.js**, which allowed users to work with a fully-featured, WYSIWYG text editor directly in the browser.

These skills are vital for building user-friendly interfaces—something that often sets great projects apart in hackathons. Now, I can rapidly prototype clean, responsive UIs without relying on heavier frontend frameworks. It also enables me to build more complex user interactions such as case editors, file managers, and real-time document viewers in future applications.

✅ 2. Deepening Flask Knowledge

I significantly expanded my understanding of Flask during this project. I learned to manage user sessions and handle authentication using **Flask-Login** in combination with **Werkzeug** for security. I also structured a scalable backend using Flask routes for tasks like document analysis, file handling, and dashboard management. Additionally, I became more proficient in using **Jinja2** to dynamically render HTML templates with conditionals and loops based on user data.

This deeper understanding of Flask now gives me the ability to create full-featured, secure web applications from scratch. In a hackathon environment, this translates to faster development and better backend structure, which are critical for time-sensitive projects.

✅ 3. Secure User Authentication (Werkzeug)

Security is a foundational aspect of any application, and I took care to implement secure authentication using **Werkzeug**. I learned how to securely hash passwords and validate user credentials, as well as how to manage sessions and ensure that users stay logged in securely across different pages.

These are essential skills for any app that handles private data, and I now feel equipped to build secure authentication systems for user portals, dashboards, or any collaborative platforms in future work.

✅ 4. File Handling with python-docx

This project marked my first experience with **python-docx**, a library that allowed me to extract text and structure from `.docx` files uploaded by users. I learned how to parse Word documents programmatically, retrieve content for AI analysis, and present the results in an interactive format.

This opens up a whole new set of possibilities in future projects. I can now build applications that deal with resumes, legal documents, case files, or essays—allowing users to upload documents and receive intelligent feedback or transformations in real time.

✅ 5. API Integration with OpenRouter

Integrating external APIs was another major milestone. I learned to send structured requests to **OpenRouter**, handle JSON responses, and inject the AI-generated feedback into my application's frontend. This created a powerful user experience where content could be analyzed, summarized, and improved with just one click.

In today's tech landscape, API integration is essential. Whether it's AI, payment gateways, or social media platforms, APIs are everywhere. Having hands-on experience with OpenRouter gives me a competitive edge and prepares me to work with more sophisticated APIs like OpenAI, Cohere, Stripe, or Firebase in the future.

Final Thoughts

This project was far more than just a technical exercise—it was a full-stack development journey that pushed me to grow as both a frontend and backend developer. I learned to design elegant interfaces, build secure and scalable backend systems, and integrate advanced AI technologies. These aren't just academic skills—they're practical tools I can now bring to future hackathons, collaborative tech competitions, freelance work, or personal startup ideas.

By mastering these tools and frameworks, I now feel better prepared to handle complex development tasks, lead technical projects, and build applications that are not only functional but also visually compelling and impactful. This project has truly leveled up my development capabilities.